Hierarchical Transformer Preconditioning for Real-Time Simulation

ANONYMOUS AUTHOR(S)
SUBMISSION ID: 0000

We present a hierarchical transformer-based neural preconditioner for the large, sparse linear systems that arise in real-time physics simulation. Unlike prior learning-based preconditioners that either impose sparsity patterns inherited from incomplete factorizations or rely on purely local graph convolutions, our method learns an approximation to the full matrix inverse structured by an analytically constructed, weak-admissibility $\mathcal{H}$-matrix partition. On the diagonal, full-rank symmetric factors $UU^\top$ are produced per leaf block, capturing high-frequency local interactions without rank restriction. Off the diagonal, a scale-matched low-rank factorization $UV^\top$ is produced at a fixed coarse resolution independent of block separation, encoding low-frequency long-range couplings with a rank budget that naturally shrinks as distance from the diagonal grows—matching the spectral structure of simulation residuals. All blocks are processed by two compact transformer stacks operating on contiguous memory, eliminating the pointer-chasing and triangular-solve bottlenecks that limit GPU utilization in competing approaches. The result is an $O(N)$ preconditioner with substantially higher arithmetic intensity than graph-neural-network baselines, fully differentiable and trained end-to-end via a cosine-similarity Hutchinson probe loss. We demonstrate real-time preconditioner inference and significantly accelerated PCG convergence across a range of simulation problem scales and material configurations.

CCS Concepts: • **Computing methodologies** → **Physical simulation**.

Additional Key Words and Phrases: preconditioning, iterative solvers, hierarchical matrices, transformers, neural operators, physics simulation, real-time graphics

ACM Reference Format:
Anonymous Author(s). 2026. Hierarchical Transformer Preconditioning for Real-Time Simulation. *ACM Trans. Graph.* 45, 6, Article 1 (December 2026), 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Real-time simulation of fluids, elastic bodies, and coupled multi-physics systems requires solving a large, sparse linear system $A\mathbf{x} = \mathbf{b}$ at every time step, typically via preconditioned conjugate gradient (PCG). The preconditioner is the dominant design variable: it must be cheap to apply, yet must dramatically cluster the eigenvalues of MA near unity to reduce the number of PCG iterations. In interactive and real-time settings the per-frame budget is measured in milliseconds, ruling out methods such as incomplete Cholesky (IC) or algebraic multigrid (AMG), whose construction costs are either unpredictable or prohibitively expensive, and whose application involves sequential triangular solves that are notoriously hard to parallelize on modern GPUs [Yang et al. 2025].

Machine learning offers an appealing alternative: given a distribution of simulation states, a neural network can be trained once and then applied at near-constant, predictable cost. Graph neural networks (GNNs) have emerged as the dominant architecture for this purpose [Chen 2024; Häusner et al. 2023; Li et al. 2023; Yang et al. 2025], because the natural graph interpretation of a sparse matrix allows message-passing layers to propagate information through the nonzero structure. GNN-based preconditioners have shown impressive results on ill-conditioned problems where classical algebraic methods fail [Chen 2024].

Nevertheless, GNN-based approaches carry fundamental limitations that become acute in the simulation setting. They impose strict sparsity constraints: the preconditioner's nonzero pattern is determined by the sparsity of A, meaning non-adjacent interactions are either ignored entirely or approximated only through repeated graph convolutions. Because information about node i reaches node j only via graph paths, effective conditioning of long-range couplings requires many GNN layers, which either smooth away local information [Wu et al. 2019] or require expensive skip connections. GNN evaluation is further dominated by irregular memory access patterns and scatter-gather operations, leading to low arithmetic intensity and poor GPU utilization. Prior factorization-based approaches [Häusner et al. 2023; Li et al. 2023] produce triangular factors whose application requires sequential substitution—a task that maps poorly to the massively parallel architecture of modern GPUs [Yang et al. 2025].

In this paper we propose a fundamentally different architecture that addresses all of these limitations simultaneously. Our key observation is that simulation matrices have a structured connection between *spatial proximity* and *spectral content* that can be exploited to introduce a strong, analytically grounded structural bias into the neural preconditioner.

Spatial ordering and spectral content. Simulation codes routinely reorder their degrees of freedom using space-filling curve orderings such as Morton (Z-curve) or bandwidth-reduction orderings such as reverse Cuthill–McKee (RCM), to maximize cache locality and support spatial hashing. Under such orderings, near-diagonal entries of A correspond to interactions among spatially nearby particles or mesh elements, while far-diagonal entries correspond to spatially distant interactions. This is the well-known $\mathcal{H}$-matrix-compatible banded structure [Hackbusch 1999].

A classical observation in hierarchical matrix theory and multigrid methods is that residual errors are spectrally structured: high-frequency errors (oscillatory residual components) arise from near-field interactions and require locally supported corrections, while low-frequency errors—a bulk pressure shift, a DC temperature mode, a large-scale stress wave propagating from a force application site to a grounding boundary—arise from long-range interactions and require globally informed corrections. Because near-diagonal entries encode high-frequency near-field interactions and far-diagonal entries encode low-frequency far-field interactions, the banded structure of A maps precisely onto this spectral hierarchy. This alignment is the key structural insight our method exploits.

Our approach. We partition A into a *weak-admissibility $\mathcal{H}$-matrix* decomposition, computed analytically and held fixed throughout training and inference. Given a minimum block size L, the partition consists of $K = \lceil N/L \rceil$ dense diagonal blocks of size $L \times L$, and a set

2026. ACM 1557-7368/2026/12-ART1
https://doi.org/10.1145/nnnnnnn.nnnnnnn

of off-diagonal tiles each of size $S \times S$, where S grows as a power of two with distance from the diagonal.

For the diagonal blocks, the network outputs a factor $F \in \mathbb{R}^{L_s \times L_s}$ (with $L_s = L/2$) and the approximate inverse block is $FF^\top$—symmetric positive semidefinite with full rank L_s, providing a rank-unrestricted local approximation that captures high-frequency errors without rank restriction.

For each off-diagonal block of original size $S \times S$, the network produces two thin factor matrices $U_m, V_m \in \mathbb{R}^{L_s \times L_s}$. Crucially, *all off-diagonal blocks are processed at the same downsampled resolution* $L_s \times L_s$, regardless of S. The $S \times S$ region of the approximate inverse is assembled via node projection factors that expand the coarse representation back to full node resolution (Section 4.7). The effective rank of this approximation is at most L_s, so as S grows with distance from the diagonal, the rank-to-size ratio L_s/S decreases—precisely the right structural bias for low-frequency, long-range couplings.

Because all off-diagonal blocks share the same token count (L_s), and the total number of off-diagonal blocks is $M_{\mathcal{H}} = O(N/L)$, both the preconditioner construction (a single forward pass) and application (batched matrix multiplications) scale as $\Theta(N)$—not $O(N \log N)$ as in classical $\mathcal{H}$-matrix schemes.

The resulting system offers four specific advantages over prior neural preconditioners. *Full coverage*: it explicitly preconditions non-adjacent long-range interactions by design, unlike sparse-pattern or GNN-based approaches. *No triangular solves*: application requires only batched matrix multiplications, mapping directly to GPU Tensor Cores. *Hardware alignment*: the two transformer stacks performing dense batched tensor contractions are precisely the workload for which modern GPUs and their successors are optimized, aligning with the trajectory of accelerator hardware rather than working against it. *Predictable cost*: the static $\mathcal{H}$-matrix partition means zero data-dependent branching and completely predictable FLOP count, enabling hard real-time deployment.

2 Related Work

2.1 Classical Iterative Solvers and Preconditioners

Preconditioning large, sparse linear systems is a cornerstone of scientific computing [Saad 2003]. Widely used algebraic preconditioners include Jacobi (diagonal scaling), incomplete LU and Cholesky factorizations (ILU/IC) [Lin and Moré 1999], sparse approximate inverse (SPAI) [Benzi et al. 1996; Kolotilina and Yeremin 1993], and algebraic multigrid (AMG) [Ruge and Stüben 1987]. Each involves a fundamental tension between approximation quality and computational cost. IC and ILU achieve strong spectral approximations but require triangular solves whose sequential dependence structure resists GPU parallelization [Liu et al. 2016; Yamazaki et al. 2020]. SPAI methods avoid triangular solves but their construction requires sequential optimization and often produces suboptimal condition numbers [Yang et al. 2025]. AMG offers excellent convergence for elliptic PDEs but requires complex, sensitive setup and irregular V-cycle communication patterns [Naumov et al. 2015]. In real-time simulation, the construction cost of IC and AMG is frequently the dominant bottleneck; our training-amortized approach provides constant-cost construction at inference time.

2.2 Neural Operators and Machine Learning for Physics

Neural operators including FNO [Li et al. 2021], DeepONet [Lu et al. 2021], and graph-based variants [Li et al. 2020a,b] learn mappings between function spaces and have been applied as PDE surrogates and as approximate solvers. FCG-NO [Rudikov et al. 2024] embeds neural operators as preconditioners inside flexible Krylov solvers. Physics-informed neural networks (PINNs) [Raissi et al. 2019] incorporate PDE residuals directly into the loss. Our work is complementary: we operate algebraically, at the level of the assembled sparse matrix A and simulation-state node features, without reference to an underlying PDE or coordinate system beyond node positions. This generality means our preconditioner applies to any large sparse system from the simulation, including those with non-uniform material parameters, complex geometries, or irregular discretizations.

2.3 Learning-Based Preconditioners

An important line of work trains GNNs to predict the nonzero entries of an incomplete Cholesky or LU factor [Häusner et al. 2023; Li et al. 2023; Trifonov and Güttel 2024], replacing sequential fill-in computation with a parallel GNN forward pass. These methods can produce higher-quality factors than classical IC, but inherit its triangular-solve bottleneck at application time, and require GNN architectures to implicitly model the global elimination tree—a long-range dependency that local message passing is theoretically ill-suited to capture [Trifonov et al. 2025].

Yang et al. [Yang et al. 2025] address the GPU bottleneck directly by training a GNN to output a sparse approximate inverse ($GG^\top$) rather than a triangular factor, reducing each preconditioner application to sparse matrix-vector products. Their scale-invariant SAI loss achieves effective training without needing solution vectors. Our work extends this direction: rather than constraining to the sparsity pattern of A, we structure the preconditioner as an $\mathcal{H}$-matrix that explicitly represents long-range couplings in hierarchically compressed form.

Chen [Chen 2024] proposes a GNN-based preconditioner evaluated on over 800 matrices from the SuiteSparse benchmark, demonstrating strong performance on ill-conditioned problems— particularly cases where ILU and AMG fail. Li et al. [Li et al. 2023] focus on PDE-derived FEM matrices, exploiting data distributions for improved convergence. Our target is real-time physics simulation, where the matrix A evolves at interactive rates and both construction and application costs must be minimized simultaneously. We additionally exploit rich simulation-state features—positions, velocities, physical parameters—that pure-algebraic preconditioners cannot access.

Hierarchical matrix techniques [Hackbusch 1999; Hackbusch and Khoromskij 2002] approximate data-sparse matrices by exploiting low-rank structure in well-separated sub-blocks. We draw on this formalism to define the block structure of our preconditioner, but replace the classical analytical low-rank approximation with learned transformer-generated factors, allowing the model to discover which features of the simulation state determine the rank and content of each block.

3 Background

3.1 The Conjugate Gradient Method and Preconditioning

We seek $\mathbf{x} \in \mathbb{R}^N$ satisfying $A\mathbf{x} = \mathbf{b}$, where $A \in \mathbb{R}^{N\times N}$ is symmetric positive definite (SPD). The preconditioned conjugate gradient (PCG) algorithm introduces a preconditioning operator $M \approx A^{-1}$ and at each iteration applies M to the current residual to form the search direction. Convergence is governed by the condition number $\kappa(MA)$: if all eigenvalues of MA are clustered near 1, PCG converges in few iterations. For simulation applications involving nonsymmetric or indefinite systems, flexible GMRES (FGMRES) [Chen 2024; Saad 1993] extends the framework to nonlinear preconditioners; our architecture applies equally in this setting.

3.2 $\mathcal{H}$-Matrices and Weak Admissibility

Let N degrees of freedom be partitioned into $K = N/L$ contiguous *leaves* of size L, indexed $0, \ldots, K-1$ in the order induced by a spatial curve or bandwidth-reduction reordering. A block connecting leaves with index ranges $\mathcal{I}$ and $\mathcal{J}$ is *weakly admissible* if

$$|\mathcal{I}| \le \eta \cdot \mathrm{dist}(\mathcal{I}, \mathcal{J}),$$

where $\mathrm{dist}(\mathcal{I}, \mathcal{J}) = \min_{i\in\mathcal{I}, j\in\mathcal{J}} |i - j|$ is the gap in index space. An $\mathcal{H}$-matrix approximation retains diagonal blocks at full rank and replaces each weakly admissible off-diagonal block with a low-rank factorization $UV^\top$.

In our implementation (`hmatrix.py`), the partition is computed by assigning to each off-diagonal leaf pair (k, l) the largest admissible block size $S = 2^j$ (in units of L) satisfying the admissibility criterion, and collecting the unique tiles. The number of off-diagonal blocks satisfies $M_{\mathcal{H}} \approx c_\eta K$ for a small constant c_η (approximately 9 for $\eta = 1$, $K = 16$), so total storage and computation remain $O(N)$. The partition is computed once at model initialization and held fixed for all inputs.

3.3 Hutchinson Trace Estimation

The Hutchinson estimator [Hutchinson 1989] provides an unbiased stochastic approximation:

$$\mathrm{tr}(B) = \mathbb{E}_{\mathbf{z}\sim\mathcal{N}(0,I)}\left[\mathbf{z}^\top B\mathbf{z}\right].$$

We use this to measure preconditioner quality without explicit access to A^{-1}: by drawing probe vectors $\mathbf{z}$, computing $A\mathbf{z}$ via sparse matrix-vector products, applying M to obtain $\widehat{\mathbf{z}} = M(A\mathbf{z})$, and measuring the alignment of $\widehat{\mathbf{z}}$ with $\mathbf{z}$, we obtain a differentiable scalar loss whose gradient drives the network toward $MA \approx I$. The cosine formulation (Section 6.1) makes this loss invariant to the overall scale of A, matching the scale-invariance of PCG convergence rates.

4 Method

4.1 Overview

Our preconditioner is produced by the LeafOnlyNet architecture, a neural network f_θ mapping a simulation state and sparse matrix A to a packed tensor encoding all factors of the $\mathcal{H}$-matrix approximate inverse. The architecture has four stages: (1) a **physics-aware encoder** lifting raw node features to rich d-dimensional embeddings via a small GCN operating on the full graph structure of A; (2) a **diagonal transformer stack** processing one token per leaf block, attending densely within each block to capture local high-frequency structure; (3) an **off-diagonal transformer stack** that, within each off-diagonal $\mathcal{H}$-block, assigns one token per downsampled group of nodes at fixed coarse resolution, capturing long-range low-frequency couplings; and (4) **linear decoder heads** projecting token embeddings to factor matrices. Optionally, **highway connections** couple the two stacks at each layer, propagating row-band and column-band context without breaking the $O(N)$ computation budget.

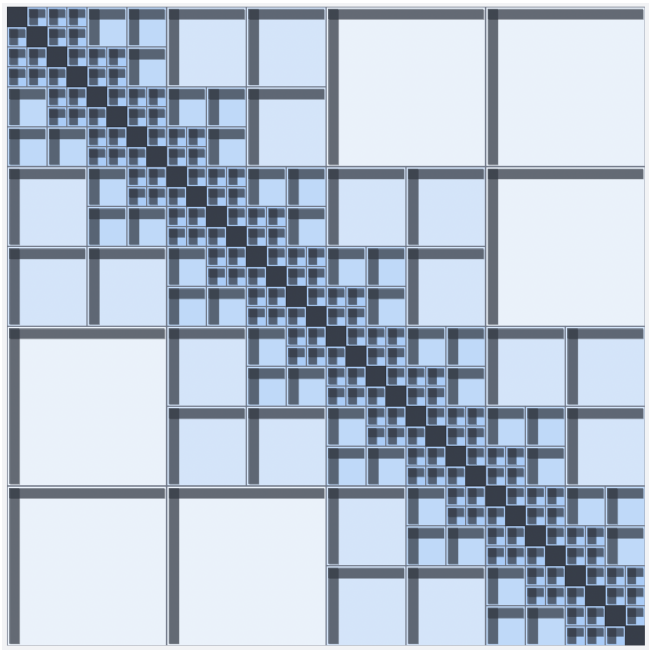

Fig. 1. Schematic of a weak-admissibility $\mathcal{H}$-matrix partition on a uniform leaf grid (illustrative leaf count). Dense diagonal blocks appear as small squares along the main diagonal; admissible off-diagonal tiles grow with separation from the diagonal. A translucent, desaturated red overlay marks the upper-triangle portion (including the diagonal) of the row- and column-strip leaf index sets for one exemplar off-diagonal tile near the grid center (the same index sets accumulated into row/column token highways in training), drawn on top of the usual blue tile tint. Figure produced with `hmatrixgraphic.py`.

4.2 The $\mathcal{H}$-Matrix Partition

Given N nodes ordered by a spatial curve and minimum block size L, we define $K = N/L$ leaves. Figure 1 sketches the resulting tile layout (schematic). The weak-admissibility partition is computed analytically by `standard_admissible_unique_blocks` in `hmatrix.py`, which vectorizes the admissibility condition over the full $K \times K$ leaf grid. The result is three static integer arrays—row origin $r_0 \in \mathbb{Z}^{M_{\mathcal{H}}}$, column origin $c_0 \in \mathbb{Z}^{M_{\mathcal{H}}}$, and block size $S \in \mathbb{Z}^{M_{\mathcal{H}}}$ (in leaf units)—computed once at model initialization and never recomputed. Only blocks with $r_0 \le c_0$ are stored (strict upper triangle); the preconditioner is applied symmetrically. The partition is fully static with zero data-dependent branching, making the FLOP count and memory footprint completely predictable at runtime.

4.3 Physics-Aware Encoder

Simulation nodes carry raw features including spatial position $\mathbf{p}_i \in \mathbb{R}^3$, velocity $\mathbf{v}_i \in \mathbb{R}^3$, and additional physical quantities such as density, pressure, or material parameters. The encoder (`PhysicsAwareEmbedding`) lifts these to d-dimensional embeddings in two phases.

A two-layer MLP (`lift`) first maps raw node features (optionally augmented with a broadcast global feature vector encoding simulation-level context such as time step size or mean density) to dimension d. The embeddings then pass through $n_{\text{gcn}} = 2$ layers of a sparse GCN (`SparsePhysicsGCN`), each aggregating information from graph neighbors weighted by entries of A:

$$\mathbf{h}_i^{(\ell+1)} = \mathbf{h}_i^{(\ell)} + g\Big(\big[W_{\text{self}}\mathbf{h}_i^{(\ell)};\ \textstyle\sum_{j\in\mathcal{N}(i)} A_{ij} W_{\text{nbr}}\mathbf{h}_j^{(\ell)}\big]\Big), \qquad (1)$$

where $\mathcal{N}(i)$ denotes graph neighbors of i and g is a two-layer MLP with GELU activation. This small GCN provides each node's embedding with information about its local neighborhood in both graph structure and matrix values, as well as the physical state of nearby simulation particles. Unlike purely algebraic preconditioners, this encoder can incorporate positions, velocities, and material parameters invisible to IC, AMG, or SPAI.

The GCN is the only part of the architecture that directly sees individual graph edges; all subsequent processing operates on the block-partitioned representation. The encoder output $\mathbf{h} \in \mathbb{R}^{B\times N\times d}$ is projected by a linear layer (`enc_input_proj`) before being distributed to the two transformer streams.

4.4 Diagonal Transformer Stack

The diagonal stream operates on a downsampled representation of the K leaf blocks. Each block of L nodes is mean-pooled by factor p_{diag} to produce $L_s = L/p_{\text{diag}}$ tokens per block, for a total sequence length of KL_s. The diagonal stack consists of n_{layers} `TransformerBlock` modules, each combining a `LeafBlockAttention` module and a position-wise FFN.

Intra-block attention. Attention is performed entirely within each leaf block, so total attention cost is $O(KL_s^2) = O(N)$. The attention logits are augmented with a physics-derived edge bias: for each token pair (i, j) within the same block, the edge feature vector $\mathbf{e}_{ij} \in \mathbb{R}^4$ encodes $[\Delta x, \Delta y, \Delta z, A_{ij}]$—spatial displacement and matrix entry. A small MLP (`edge_gate`) maps these features to a per-head scalar bias added to the logits before softmax, allowing the model to modulate attention based on both geometric proximity and coupling strength. Diagonal entries are marked with a special flag so the model can identify self-interactions.

4.5 Off-Diagonal Transformer Stack

For the off-diagonal $\mathcal{H}$-matrix blocks, the model constructs a low-rank approximation for each block m. We first extract a *strip embedding* for each block by pooling encoder embeddings across its row and column leaf regions.

Precomputed static weight matrices $W^{\text{row}} \in \mathbb{R}^{M_{\mathcal{H}}\times K}$ and $W^{\text{col}} \in \mathbb{R}^{M_{\mathcal{H}}\times K}$ (stored in `hmatrix.py`) perform mean pooling over the appropriate leaf ranges:

$$\mathbf{s}_m = \big(W^{\text{row}}_{m,:} + W^{\text{col}}_{m,:}\big)\mathbf{H}_{\text{leaves}}, \qquad (2)$$

where $\mathbf{H}_{\text{leaves}} \in \mathbb{R}^{K\times L\times d}$ is the post-encoder diagonal representation. The sum of row and column pool weights gives each off-diagonal token an initial embedding reflecting the physical state of both regions it couples. When $p_{\text{off}} > 1$, the L tokens per strip are further mean-pooled to L_s tokens, forming the input to the off-diagonal stack with shape $(BM_{\mathcal{H}}, L_s, d)$.

The off-diagonal stack has the same structure as the diagonal stack but its attention operates across the L_s tokens of each off-diagonal block independently. Edge features for off-diagonal attention are the mean edge features $[\Delta x, \Delta y, \Delta z, A_{ij}]$ aggregated over all node pairs (i, j) with i in the row region and j in the column region sharing a graph edge, capturing the typical coupling geometry and strength.

The key insight of this design is that every off-diagonal block is processed at the same token resolution L_s, regardless of the physical separation or size of the regions it connects. A block coupling two regions of 512 nodes each is processed with exactly the same L_s-token representation as one coupling two regions of 128 nodes. The difference emerges in the rank relative to block size: the 512-node coupling has effective rank fraction $L_s/512$ versus $L_s/128$ for the 128-node case. This is precisely the behavior of an ideal $\mathcal{H}$-matrix approximation: farther blocks are compressed more aggressively because they represent lower-frequency, smoother interactions.

4.6 Highway Connections

A transformer restricted to attending within its block develops rich representations of its specific interactions but remains uninformed about the global state. This is a genuine limitation: inspection of approximate inverse blocks reveals that up to 30% of the energy in any given block comes from interactions with other blocks sharing the same row or column band—couplings that a purely local block transformer cannot directly observe.

To address this, we optionally introduce *highway connections* coupling the two stacks at each transformer layer. The highway maintains a pair of N-length vectors (row and column directions) and a single global token, all of dimension d:

$$\mathbf{r}_{\text{hw}} \in \mathbb{R}^{B\times N\times d}, \quad \mathbf{c}_{\text{hw}} \in \mathbb{R}^{B\times N\times d}, \quad \mathbf{g}_{\text{hw}} \in \mathbb{R}^{B\times d}.$$

After the attention sub-layer of each transformer block, highway vectors are updated by scatter-adding the current block token embeddings into their corresponding positions in the global N-length buffers. Off-diagonal block tokens are first expanded back to full leaf resolution via `repeat_interleave` before scatter, distributing their contribution appropriately across all nodes in their row and column ranges.

Before the FFN sub-layer, each token $\mathbf{z}$ reads from the highway to form its FFN input:

$$\tilde{\mathbf{z}} = \big[\mathbf{z};\ \mathbf{r}_{\text{hw}}[\text{pos}(\mathbf{z})];\ \mathbf{c}_{\text{hw}}[\text{pos}(\mathbf{z})];\ \mathbf{g}_{\text{hw}}\big] \in \mathbb{R}^{4d}, \qquad (3)$$

which is processed by the FFN's input projection (width $4d$). Without highways, the FFN receives only $\mathbf{z} \in \mathbb{R}^d$.

The highway implements four complementary communication channels simultaneously: a *2D channel* (intra-block attention, spatially local), two *1D axial channels* (row and column highways propagating information along entire matrix bands), and a *0D global channel* (the broadcast mean token carrying system-wide context). This 2D/1D/1D/0D multiscale structure arises from the physics of

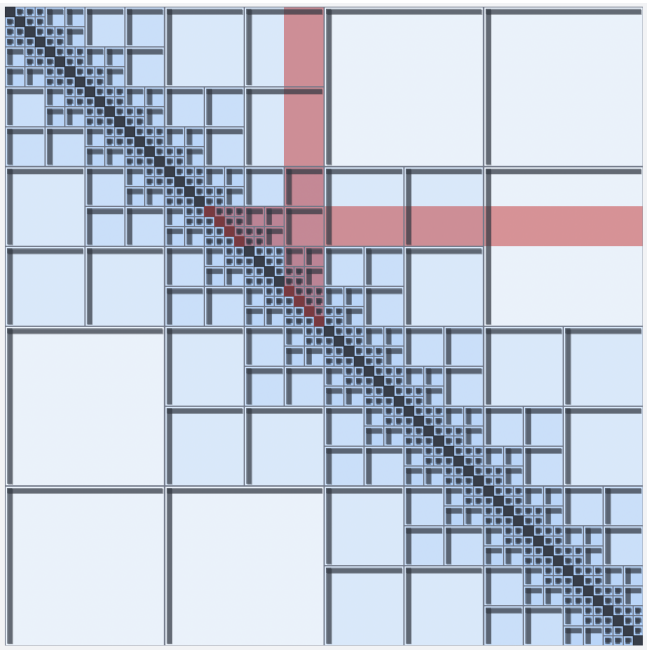

Fig. 2. Leaf-grid schematic of the same weak-admissibility partition as Figure 1, with translucent red highlighting the upper-triangle portion (including the diagonal) of the row- and column-strip leaf index sets accumulated from one exemplar off-diagonal tile—the same index pattern written by prolongation into the row and column highway buffers during training. Figure produced with `hmatrixgraphic.py` (`SHOW_HIGHWAYS=True`).

the problem: the approximate inverse of a banded matrix has precisely this nested band-row-column-global structure. The highway adds only $O(Nd)$ memory and computation per layer, preserving the overall $O(N)$ scaling.

4.7 Decoder and Output Structure

After the two transformer stacks, lightweight linear decoder heads project the final token embeddings to preconditioner factors.

Diagonal blocks. The diagonal stream output is viewed as K groups of L_s tokens. A two-layer MLP (`leaf_head`) projects each token to $\mathbb{R}^{L_s}$, giving a factor matrix $F_k \in \mathbb{R}^{L_s \times L_s}$ per leaf block. The diagonal block of the approximate inverse for leaf k is $F_k F_k^\top \in \mathbb{R}^{L_s \times L_s}$, symmetric positive semidefinite by construction.

Off-diagonal blocks. Two heads (`off_diag_head_U`, `off_diag_head_V`) project d-dimensional tokens to $\mathbb{R}^{L_s}$, producing factor matrices $U_m, V_m \in \mathbb{R}^{L_s \times L_s}$ for each off-diagonal block m. The block of the approximate inverse is $U_m V_m^\top$ above the diagonal and $V_m U_m^\top$ below, maintaining global symmetry.

Node projection factors. To expand the coarse L_s-dimensional representation back to full node resolution for preconditioner application, two linear heads (`node_u`, `node_v`) project each full-resolution node embedding (obtained by upsampling the diagonal stream via `repeat_interleave`) to $\mathbb{R}^{L_s}$. For off-diagonal block m connecting regions of size S each, the resulting node factor matrices $\tilde{U}_m, \tilde{V}_m \in \mathbb{R}^{S \times L_s}$ allow FMM-style application: applying the block to a vector costs three matrix multiplications of sizes $S \times L_s$, $L_s \times L_s$, and $L_s \times S$, i.e., $O(SL_s)$ per block and $O(NL_s)$ in total since $\sum_m S^{(m)} = O(N)$.

Jacobi residual. A scalar gate (`jacobi_gate`) produces a learned diagonal correction $\lambda_i \in \mathbb{R}$ per node, allowing the model to add a Jacobi-like term $\mathrm{diag}(\boldsymbol{\lambda}) \cdot \mathrm{diag}(A)^{-1}$ to the preconditioner output as a simple residual path when the hierarchical factors underfit. During early training, when those factors still approximate the inverse poorly, gradients tend to rely more heavily on this cheap diagonal correction so that the Hutchinson loss decreases before the full $\mathcal{H}$-structure has fully fit; as the learned factors improve, the gate can reduce the Jacobi contribution so capacity shifts toward the hierarchical representation.

Packed output. All factors are flattened and concatenated into a single packed tensor $\mathbf{p} \in \mathbb{R}^{B \times P}$ with $P = KL_s^2 + M_{\mathcal{H}} L_s^2 + 2NL_s + N$, consumed by the application routine (`apply_block_diagonal_M`).

4.8 Complexity Analysis

Let L be the leaf size, $K = N/L$, $L_s = L/2$, d the embedding dimension, and n_l the number of layers.

Encoder: $O(Nd^2)$ for bounded-degree simulation graphs.

Diagonal stack: Per-block attention $O(L_s^2 d)$ over K blocks and n_l layers: $O(n_l K L_s^2 d) = O(N)$. FFN: $O(n_l K L_s d^2) = O(N)$.

Off-diagonal stack: $M_{\mathcal{H}} = O(K) = O(N/L)$ blocks with L_s tokens each. Per-block attention $O(L_s^2 d)$; total $O(M_{\mathcal{H}} L_s^2 d) = O(Nd)$. FFN: $O(Nd^2/L)$.

Highway: Scatter-gather: $O(Nd)$ per layer.

Application: Diagonal blocks: $O(KL_s^2) = O(NL_s/L)$. Off-diagonal FMM chain: $O(\sum_m S^{(m)} L_s) = O(NL_s)$. Total: $O(N)$.

Both the preconditioner construction (one LeafOnlyNet forward pass) and application are $\Theta(N)$, with constants determined by fixed hyperparameters d, L, L_s, n_l.

5 Preconditioner Application

5.1 Matrix-Free Operator Application

At each PCG iteration the solver requires one application of the preconditioner M to the current residual $\mathbf{r}$, producing the search direction $\mathbf{z} = M\mathbf{r}$. A critical design choice, shared with sparse approximate inverse methods [Kolotilina and Yeremin 1993; Yang et al. 2025], is that M is *never explicitly instantiated as a dense or sparse matrix.*

To see why, consider the alternative. The full approximate inverse $M \in \mathbb{R}^{N \times N}$ has $O(N^2)$ entries in the worst case. Even under the $\mathcal{H}$-matrix sparsity structure, assembling the $S \times S$ off-diagonal blocks (which can be as large as $N/2 \times N/2$ for the outermost tiles) and then performing the matrix-vector product $M\mathbf{r}$ as a general sparse or dense multiplication would require $O(N^2)$ storage and $O(N^2)$ FLOPS— quadratic in N. More practically, forming M explicitly and then computing AM to work in the "left-preconditioned" space $AM\mathbf{u} = \mathbf{b}$ would destroy the factored low-rank structure of the off-diagonal blocks and would require materializing product matrices of size $N \times N$, which is infeasible for the system sizes we target.

Instead, we work in the standard *right-preconditioned* PCG formulation: the solver maintains the unpreconditioned residual $\mathbf{r} = \mathbf{b} - A\mathbf{x}$,

and at each iteration applies M to $\mathbf{r}$ in operator form by evaluating the sequence of tensor contractions described below. This is identical in structure to the application of SPAI preconditioners via sparse matrix-vector products [Yang et al. 2025], but with a hierarchical factored operator rather than an explicit sparse matrix. The full $N \times N$ structure of M is never realized in memory; only the packed factor tensor $\mathbf{p} \in \mathbb{R}^P$ (with $P \ll N^2$) is stored, and the application of M to any given vector costs $O(N)$ in both time and temporary memory.

5.2 Diagonal Block Application

The diagonal contribution is the simplest and most regular step. Given the packed factor tensor, the diagonal blocks are extracted as a strided view—no copy—into the contiguous segment $\mathbf{p}[: KL_s^2]$, yielding a tensor $F \in \mathbb{R}^{K \times L_s \times L_s}$ of K small square matrices. The input residual $\mathbf{r} \in \mathbb{R}^N$ is reshaped (zero-copy) into K leaf-local segments of length L each.

If $p_{\text{diag}} = 1$ (no pooling), the application is a single batched matrix-vector product:

$$\mathbf{y}_k^{\text{diag}} = F_k \, \mathbf{r}_k, \quad k = 0, \ldots, K-1, \tag{4}$$

executed as `torch.matmul(diag_blocks, x_leaves)` with shapes $(K, L_s, L_s) \times (K, L_s, 1)$, which dispatches to a single batched GEMM call on the GPU. When $p_{\text{diag}} > 1$, the input is first mean-pooled within each leaf ($L \to L_s$ tokens, shape $(K, L_s, p_{\text{diag}}, 1) \to (K, L_s, 1)$) and the output is upsampled by `repeat_interleave` back to length L—two elementwise operations bracketing the GEMM.

The batched GEMM is the dominant cost of the diagonal step: K multiplications of $L_s \times L_s$ matrices by L_s-vectors, totaling $2KL_s^2 = 2NL_s/L$ FLOPs, which is $O(N)$. Crucially, all K blocks are stored contiguously in memory, so the GPU sees a single large GEMM operand rather than K scattered pointer lookups. This is the key difference from block-sparse ILU or block-Jacobi preconditioners whose irregular sparsity forces separate kernel launches or gather operations per block; here the block structure is regular by construction. The arithmetic intensity (FLOPs per byte of memory traffic) is $L_s/2$, much higher than the scalar diagonal multiply of Jacobi preconditioning ($L_s/2$ vs. 1 FLOP/8 bytes) and comparable to a dense matrix-vector product at the scale of one leaf block.

5.3 Off-Diagonal Block Application: FMM-Style Multi-Stage Pipeline

The off-diagonal contribution follows a Fast Multipole Method (FMM) [Greengard and Rokhlin 1987]-style multi-stage pipeline: *projection* (restriction to coarse moments), *strip aggregation*, *coarse interaction* (the batched small-matrix multiply), and *prolongation* (expansion back to full node resolution). All intermediate buffers are preallocated once at solver setup by `block_diagonal_m_apply_workspace` and reused across all PCG iterations; no dynamic allocations occur inside the hot loop.

Stage 1: Node projection (restriction). The node factor matrices $\tilde{U} \in \mathbb{R}^{K \times L \times L_s}$ and $\tilde{V} \in \mathbb{R}^{K \times L \times L_s}$ (decoded per-node from the post-encoder embeddings and stored in the packed tensor) project the full-resolution input into coarse "moment" vectors, one per leaf per latent dimension:

$$\hat{\mathbf{u}}_k = \tilde{U}_k^\top \mathbf{r}_k \in \mathbb{R}^{L_s}, \quad \hat{\mathbf{v}}_k = \tilde{V}_k^\top \mathbf{r}_k \in \mathbb{R}^{L_s}, \quad k = 0, \ldots, K-1. \tag{5}$$

In code this is `torch.matmul(node_U.transpose(-1,-2), x_leaves)`, yielding tensors of shape $(K, L_s, 1)$—again a single batched GEMM, $2KLL_s$ FLOPs.

Stage 2: Strip aggregation. The coarse moments $\hat{\mathbf{u}}_k$ are then aggregated into *row strip* and *column strip* vectors for each off-diagonal block m using the precomputed static weight matrices $W^{\text{row}}, W^{\text{col}} \in \mathbb{R}^{M_{\mathcal{H}} \times K}$ (binary sum-pool weights stored in `HM_SUM_W_ROW_CPU` / `HM_SUM_W_COL_CPU`):

$$\mathbf{s}_m^{\text{row}} = \sum_{k : r_0^{(m)} \le k < r_0^{(m)} + S^{(m)}} \hat{\mathbf{u}}_k, \quad \mathbf{s}_m^{\text{col}} = \sum_{k : c_0^{(m)} \le k < c_0^{(m)} + S^{(m)}} \hat{\mathbf{v}}_k. \tag{6}$$

In the allocation-free inference path (`apply_block_diagonal_m_into`), this is implemented as two `torch.index_select` calls (gathering the leaf entries participating in each block's row and column range, indexed by the precomputed `HM_PROLONG_ROW_LEAF_IDX` and `HM_PROLONG_COL_LI` arrays) followed by two `index_add_` operations that accumulate the gathered entries into the per-block strip buffers. Both index arrays are precomputed once at module initialization from the static $\mathcal{H}$-matrix partition and pinned to the GPU; no index computation occurs at runtime.

Stage 3: Coarse interaction. The core of the off-diagonal application is a single batched matrix-vector product between the $M_{\mathcal{H}}$ coarse blocks $B_m \in \mathbb{R}^{L_s \times L_s}$ and the strip vectors:

$$\mathbf{t}_m^{\text{row}} = B_m \, \mathbf{s}_m^{\text{col}}, \tag{7}$$

$$\mathbf{t}_m^{\text{col}} = B_m^\top \mathbf{s}_m^{\text{row}}, \tag{8}$$

for $m = 0, \ldots, M_{\mathcal{H}} - 1$. Both multiplications are dispatched as `torch.bmm(Br, xc)` and `torch.bmm(Br.transpose(-1,-2), xr)` on the reshaped $(M_{\mathcal{H}}, L_s, L_s)$ block tensor and $(M_{\mathcal{H}}, L_s, 1)$ strip vectors. This is the highest-arithmetic-intensity operation in the entire application: $M_{\mathcal{H}}$ dense $L_s \times L_s$ matrix-vector products executed in a single batched GEMM, fully occupying GPU warp schedulers with no irregular memory access. Total FLOPs: $4M_{\mathcal{H}}L_s^2 = O(NL_s)$. The symmetry of the full preconditioner ((8) uses $B_m^\top$ rather than a separately stored lower-triangular block) means only the strict upper triangle of off-diagonal blocks is stored; no additional memory is needed for the symmetric contribution.

Stage 4: Prolongation (scatter back). The coarse interaction outputs $\mathbf{t}_m^{\text{row}}$ and $\mathbf{t}_m^{\text{col}}$ are scattered back from block-indexed space to leaf-indexed space via two further `index_add_` operations using the precomputed gather index `HM_PROLONG_GATHER_IDX`, accumulating into preallocated leaf-indexed buffers of shape $(K_{\max}, L_s)$. The result is then unprojected back to full node resolution by the node factors:

$$\Delta\mathbf{y}_k = \tilde{U}_k \, \mathbf{t}_k^{\text{row,leaf}} + \tilde{V}_k \, \mathbf{t}_k^{\text{col,leaf}}, \tag{9}$$

again a batched GEMM of shape $(K, L, L_s) \times (K, L_s, 1)$, costing $4KLL_s$ FLOPs. The result $\Delta\mathbf{y}$ is added in-place to the output buffer via `out.add_(y_r_final + y_c_final)`, completing the off-diagonal contribution with no temporary tensor allocation.

Jacobi residual. If the Jacobi gate is active, a final fused multiply-add `out.addcmul_(x, (jacobi_scale * jacobi_inv_diag).unsqueeze(-1))` adds the learned diagonal correction, costing $O(N)$ elementwise operations.

5.4 Allocation-Free CUDAGraph Integration

For real-time deployment, the entire per-iteration PCG step—one CSR sparse matrix-vector product $A\mathbf{r}$ for the residual update, one call to `apply_block_diagonal_m_into` for the preconditioner application, and the PCG scalar operations (dot products, vector updates)—is captured as a CUDA Graph (`torch.cuda.graph`). CUDA Graph capture records the exact GPU kernel launch sequence during a warmup iteration and replays it on subsequent iterations with zero CPU dispatch overhead: there are no Python interpreter calls, no kernel argument serialization, and no CUDA API calls other than a single `cudaGraphLaunch`.

This is possible precisely because `apply_block_diagonal_m_into` makes *no dynamic tensor allocations* inside its body. Every intermediate buffer—`x_proj_U_flat`, `x_proj_V_flat`, `row_flat`, `col_flat`, `y_r`, `y_c`, `y_scatter_r`, `y_scatter_c`, `y_r_leaves_pad`, `y_c_leaves_pad`—is preallocated by `block_diagonal_m_apply_workspace` at solver setup and pinned for the duration of the solve. The static $\mathcal{H}$-matrix index arrays (`ridx`, `cidx`, `gidx`) are also preloaded onto the GPU before graph capture (`warmup_hmatrix_prolong_gpu`). As a result, every PCG iteration after the first executes as a single GPU kernel graph with no host-device synchronization and no memory allocation, achieving the lowest possible per-iteration overhead.

This stands in sharp contrast to IC-based preconditioners, where each application requires two sequential triangular solves whose wavefront propagation creates unavoidable data hazards between GPU threads, preventing CUDA Graph capture of a single fused solve. AMG V-cycles similarly require level-by-level synchronization barriers between coarsening stages. Jacobi preconditioning is trivially capturable but provides minimal iteration reduction. Our preconditioner is, to our knowledge, the first neural preconditioner whose application can be captured and replayed as a pure CUDA Graph with no dynamic dispatch.

5.5 Complexity and Comparison

Table 1 summarizes the per-iteration application cost of our preconditioner alongside competing methods. All FLOP counts are for one preconditioner application to a single vector; nnz denotes the number of nonzeros in A or in the relevant triangular factor.

Several comparisons deserve comment.

IC vs. ours. IC achieves $O(\text{nnz})$ FLOPs in the forward and backward substitution passes, which for bounded-degree simulation graphs is also $O(N)$. However, the forward substitution $L\mathbf{y} = \mathbf{r}$ has a strict serial dependency: entry y_i depends on y_j for all $j < i$ with $L_{ij} \neq 0$, so the GPU can only process one "level" (set of independent nodes in the elimination tree) per warp synchronization. On large matrices this wavefront scheduling underutilizes the GPU by orders of magnitude. Our method has no sequential dependencies: every GEMM kernel can be issued in parallel and all leaf blocks process simultaneously within each batched call.

Table 1. Per-iteration preconditioner application cost. N: system size. L_s: leaf apply size (our method). $K = N/L$: number of leaves. $M_{\mathcal{H}} \approx 9K$: off-diagonal blocks. nnz: nonzeros in the factor. "Sequential" indicates operations with unavoidable data-dependent wavefront scheduling that prevent full GPU parallelization.

Method	FLOPs	Memory	Parallelism
Jacobi	$O(N)$	$O(N)$	Embarrassi
IC (triangular solve)	$O(\text{nnz})$	$O(\text{nnz})$	Sequential
Neural SPAI [Yang et al. 2025]	$O(\text{nnz})$	$O(\text{nnz})$	SpMV
AMG (V-cycle)	$O(N \log N)$	$O(N \log N)$	Level-sync
Ours (diagonal)	$O(KL_s^2) = O(N)$	$O(NL_s/L)$	Batched GI
Ours (off-diag)	$O(M_{\mathcal{H}} L_s^2) = O(N)$	$O(NL_s/L)$	Batched GI
Ours (total)	$O(NL_s)$	$O(NL_s/L)$	All-GEMM

Neural SPAI vs. ours. Yang et al. [Yang et al. 2025] report that their GNN-based SPAI preconditioner requires two sparse matrix-vector products (SpMV) per PCG iteration: one for the factor G and one for $G^\top$. Each SpMV is $O(\text{nnz}(G))$ with the same sparsity pattern as A. While SpMV is GPU-friendly via CSR/BCSR formats and avoids triangular-solve sequential dependencies, it involves irregular memory access through the CSR column-index arrays—one random gather per nonzero—limiting effective arithmetic intensity. Our batched GEMMs operate on contiguous memory blocks (K or $M_{\mathcal{H}}$ dense matrices) and achieve arithmetic intensity $L_s/2$ FLOPs/byte vs. the 1 FLOP/8 bytes of a scalar SpMV. The practical consequence is that our application step saturates GPU Tensor Cores, while SpMV-based application is memory-bandwidth bound.

Furthermore, the sparsity pattern of the SPAI preconditioner in [Yang et al. 2025] is constrained to match that of A, which excludes non-adjacent couplings entirely. Our off-diagonal $\mathcal{H}$-blocks explicitly represent long-range interactions, at the cost of the additional FMM-style projection and prolongation passes—but those passes are themselves batched GEMMs with contiguous memory, not SpMVs.

Why not form M and compute AM? One might ask why we do not explicitly assemble M as a sparse matrix and then solve the left-preconditioned system $AM\mathbf{u} = \mathbf{b}$ with $\mathbf{x} = M\mathbf{u}$. There are three reasons. First, the off-diagonal blocks of M at full resolution are dense $S \times S$ matrices with S up to $N/2$: materializing them would require $O(N^2)$ storage, which is infeasible. Second, forming the product AM would densify M further through the matrix product, destroying its hierarchical structure. Third, and most importantly, the factored form $\tilde{U}_m B_m \tilde{V}_m^\top$ that we maintain allows the FMM-style application to achieve $O(N)$ cost precisely because we never form the $S \times S$ product explicitly, instead passing the low-dimensional coarse-space vector $B_m\mathbf{s}$ through the hierarchy. The matrix-free, right-preconditioned application is therefore not merely a convenience but a structural necessity for the $O(N)$ scaling guarantee.

6 Training

6.1 Objective Function

Cosine-similarity Hutchinson loss. We train LeafOnlyNet with a cosine-similarity Hutchinson probe loss. Given probe vectors

$Z \in \mathbb{R}^{N \times K_z}$, we compute AZ via sparse matrix-vector products, apply the preconditioner to obtain $\widehat{Z} = M(AZ)$, and minimize

$$\mathcal{L}_{\cos} = 1 - \frac{1}{K_z} \sum_{k=1}^{K_z} \frac{\hat{\mathbf{z}}_k \cdot \mathbf{z}_k}{\|\hat{\mathbf{z}}_k\|_2 \|\mathbf{z}_k\|_2}. \tag{10}$$

A perfect preconditioner satisfies $MA\mathbf{z} = \mathbf{z}$ for all $\mathbf{z}$, giving $\mathcal{L}_{\cos} = 0$; the worst case is $\mathcal{L}_{\cos} = 2$ (anti-aligned). Gradients flow through $M(AZ)$ into the packed preconditioner output and back through LeafOnlyNet; AZ is treated as a fixed input with no gradient through A.

Scale invariance and eigenvalue clustering. $\mathcal{L}_{\cos}$ is scale-invariant by construction: because it measures only angular alignment, $\mathcal{L}_{\cos}(MA\mathbf{z}, \mathbf{z}) = \mathcal{L}_{\cos}(\alpha MA\mathbf{z}, \mathbf{z})$ for any $\alpha > 0$. No normalization by $\|A\|$ is required, and no target scale need be specified.

This differs subtly but importantly from the SAI loss of Yang et al. [Yang et al. 2025], which penalizes $\left\| \frac{1}{\|A\|} AM^{-1} - I \right\|_{\mathrm{F}}^2$ and thereby encourages the eigenvalues of MA to cluster near 1. PCG convergence depends only on the spread of eigenvalues of MA, not their absolute location: a preconditioned system whose eigenvalues all lie in a tight cluster at $c \neq 1$ converges in as few iterations as one clustered at 1. Our loss imposes no such locality requirement. Instead, the network is free to cluster eigenvalues wherever the residual spectrum makes them easiest to align—concentrating effort on whichever frequency band the current preconditioner struggles most to reduce. In practice, we observe the network exploiting this degree of freedom adaptively: when low-frequency preconditioning is harder (as is common early in training, when off-diagonal factors are small), eigenvalues cluster at a value below 1 so that the loss does not penalize imprecise placement of the low-frequency modes; when high-frequency preconditioning is the bottleneck, the cluster drifts upward. We compare against the SAI loss and a standard ℓ_2 Hutchinson loss in the ablation study (Section 7.3), measuring both training plateau speed and the resulting eigenvalue clustering.

Spectrally biased probe vectors. Naive random probe vectors $\mathbf{z} \sim \mathcal{N}(0, I)$ weight all frequency bands uniformly by energy. In practice, however, our $\mathcal{H}$-matrix preconditioner assigns far more parameters to the diagonal blocks (one $L_s \times L_s$ factor per leaf, with no rank constraint) than to any individual off-diagonal block (one $L_s \times L_s$ factor shared across a region of up to $S \gg L_s$ nodes). With uniform probes, the gradient signal is dominated by the high-frequency, near-diagonal interactions that the diagonal blocks cover, because random vectors have most of their variation at high spatial frequency. The off-diagonal parameters, which encode low-frequency long-range structure, receive comparatively small gradients and reach saturation far later than the diagonal parameters, causing the two parts of the preconditioner to train at very different rates.

To correct this imbalance, we apply a small number of *Jacobi-smoothing* iterations to the probe vectors before use. Starting from $\mathbf{z}^{(0)} \sim \mathcal{N}(0, I)$, we iterate

$$\mathbf{z}^{(t+1)} = \mathbf{z}^{(t)} - \omega D^{-1} A \mathbf{z}^{(t)}, \tag{11}$$

where $D = \mathrm{diag}(A)$ and ω is a fixed damping parameter, for a small number of steps (`probe_z.py`). Each Jacobi step damps the high-frequency components of $\mathbf{z}$ more strongly than the low-frequency components, shifting the probe energy toward lower spatial frequencies. The smoothed probes therefore impose larger gradients on the off-diagonal parameters, which are responsible for low-frequency preconditioning, and smaller gradients on the diagonal parameters, which are already well-stimulated by high-frequency variation. The net effect is that the diagonal and off-diagonal parameters train in closer parallel, preventing the premature saturation of the diagonal factors that occurs with purely random probes. The smoothing operation is itself differentiable (it is a linear function of $\mathbf{z}^{(0)}$), but since we treat $\mathbf{z}$ as a fixed input to the loss, no gradient flows back through the smoothing steps.

6.2 Training Dynamics

Training uses AdamW with a reduce-on-plateau learning rate schedule and gradient clipping. Simulation contexts are pre-built from a set of frames and cached on disk; at each training step, a mini-batch is selected at random, padded to a common node count, and processed together. The number of probe vectors is set to $K_z = \max(64, \lceil \sqrt{N} \rceil)$ by default, balancing gradient noise and compute cost. The model is compiled with `torch.compile` for efficient GPU execution; training typically converges within tens of thousands of steps.

7 Experiments and Results

7.1 Setup

Hardware. All experiments run on a single [GPU MODEL, e.g. NVIDIA RTX 4090 / A100 80GB], CUDA [VERSION], PyTorch [VERSION], `torch.compile` with Inductor backend. PCG scalar accumulators use float64; preconditioner weights and application use float32. Inference is measured after one warmup pass with `torch.cuda.synchronize` bracketing; PCG timing uses CUDAGraph replay (Section 5.4).

Simulation dataset. Our primary benchmark is [PROBLEM TYPE, e.g. pressure Poisson from an SPH fluid simulation] with nodes ordered by [Morton / RCM]. Problem scales range from $N \in \{$[LIST, e.g. 512, 1024, 2048, 4096, 8192]$\}$. Material contrast ratio ρ is defined as [DEFINITION, e.g. ratio of max to min diagonal entry]. We train on [NUM] frames spanning [SCALE RANGE] and [CONTRAST RANGE], hold out [NUM] frames in-distribution, and reserve separate out-of-distribution splits for generalization tests.

Baselines. We compare against: unpreconditioned CG; Jacobi (diagonal scaling); IC [Lin and Moré 1999] via `ilupp` (CPU) and AMGX `MULTICOLOR_DILU` (GPU); PyAMG [Naumov et al. 2015] (CPU) and AMGX AMG (GPU); Neural SPAI [Yang et al. 2025] using their open-source checkpoint retrained on our dataset. All GPU baselines use CUDAGraph replay where supported; IC and AMG are excluded from CUDAGraph because their sequential triangular solves and V-cycle barriers prevent static capture.

Best configuration. Unless stated otherwise, results use our best-performing configuration: d =[D_MODEL], n_l =[NUM_LAYERS], L =[LEAF_SIZE], `use_highways=True`, $n_{\mathrm{gcn}} = 2$.

7.2 Ablation: Architecture Capacity and Components

We sweep architectural hyperparameters to isolate the contribution of each component. All ablation models are trained for [N] steps on [TRAINING SPLIT] at scale N =[FIXED SCALE] to control for problem difficulty.

Table 2. Ablation over architecture capacity and components at N =[FIXED SCALE]. "Inf." = inference time (ms); "Solve" = PCG solve time (ms); "Total" = Inf. + Solve; "Iters" = PCG iterations to `rtol` = 10^{-6}; "Params" = trainable parameter count. Best value per column in **bold**. All times are GPU wall-clock with CUDAGraph replay.

Configuration	Inf. (ms)	Solve (ms)	Total (ms)	Iters	Params	Train (h)
d=64, n_l=1	TODO	TODO	TODO	TODO	TODO	TODO
d=64, n_l=2	TODO	TODO	TODO	TODO	TODO	TODO
d=64, n_l=3	TODO	TODO	TODO	TODO	TODO	TODO
d=128, n_l=1	TODO	TODO	TODO	TODO	TODO	TODO
d=128, n_l=2	TODO	TODO	TODO	TODO	TODO	TODO
d=128, n_l=3	TODO	TODO	TODO	TODO	TODO	TODO
d=256, n_l=1	TODO	TODO	TODO	TODO	TODO	TODO
d=256, n_l=2	TODO	TODO	TODO	TODO	TODO	TODO
d=256, n_l=3	TODO	TODO	TODO	TODO	TODO	TODO
Best − highways	TODO	TODO	TODO	TODO	TODO	TODO
Best − GCN (n_{gcn}=0)	TODO	TODO	TODO	TODO	TODO	TODO
Best − off-diag stack	TODO	TODO	TODO	TODO	TODO	TODO
Best − Jacobi gate	TODO	TODO	TODO	TODO	TODO	TODO
Best (full)	TODO	TODO	TODO	TODO	TODO	TODO

Fig. 3. Training curves for architecture ablations at N =[FIXED SCALE]. *Left*: cosine-Hutchinson loss $\mathcal{L}_{\cos}$ vs. training step. *Right*: PCG iteration count on held-out validation frames, evaluated every 1,000 steps. The dashed line shows the equivalent SAI loss [Yang et al. 2025] value computed from the same probe vectors at each checkpoint (not used for training; shown for comparability). Horizontal grey line: Jacobi baseline iteration count (constant).

7.3 Loss Function Ablation

We compare three training losses on the best architecture: (i) our cosine-similarity Hutchinson loss $\mathcal{L}_{\cos}$ (Eq. 10); (ii) ℓ_2 Hutchinson loss $\mathcal{L}_{\ell_2} = \mathbb{E}_{\mathbf{z}}[\|MA\mathbf{z} - \mathbf{z}\|_2^2]$; (iii) the SAI loss of Yang et al. [Yang et al. 2025]. All three use the same Jacobi-smoothed probe vectors (Section 6.1).

Fig. 4. Eigenvalue distribution of MA and training dynamics for three loss functions (N =[SMALL SCALE WHERE FULL EIG IS TRACTABLE]). *Top*: eigenvalue histograms of MA for models trained with each loss, evaluated on a held-out test frame. All three models use the same architecture. *Bottom*: $\mathcal{L}_{\cos}$ on the validation set vs. training step for all three losses (common axis enables direct comparison of plateau speed).

Table 3. Loss function ablation at N =[FIXED SCALE], best architecture. "κ" = condition number of MA (median over test frames, computed via full eigendecomposition for small N, power iteration estimate for large N); "EV spread" = $(\lambda_{\max} - \lambda_{\min})/\lambda_{\text{median}}$ of MA eigenvalues; "EV location" = λ_{median} of MA; "Iters" = PCG iterations. Plateau step = training step at which $\mathcal{L}_{\cos}$ first reaches [TARGET VALUE] on validation (use $\mathcal{L}_{\cos}$ for all three to compare on a common axis).

Loss	$\kappa(MA)$	EV spread	EV location	Iters	Plateau s
$\mathcal{L}_{\ell_2}$	TODO	TODO	TODO	TODO	TO
SAI [Yang et al. 2025]	TODO	TODO	TODO	TODO	TO
$\mathcal{L}_{\cos}$ (ours)	TODO	TODO	TODO	TODO	TO

Fig. 5. Scalability: total solve time (Setup + PCG) vs. N on [PROBLEM TYPE] (log–log scale). Dashed reference line shows $O(N)$ slope. Error bars show 95% CI over [NUM] test frames.

Fig. 6. Convergence histories ($\|A\mathbf{x} - \mathbf{b}\|/\|\mathbf{b}\|$ vs. iteration) on [NUM] representative test frames at N =[MEDIUM SCALE].

7.4 Performance vs. Existing Methods

Using the best architecture (Table 2) and $\mathcal{L}_{\cos}$ (Table 3), we compare against all baselines across problem scales.

7.5 Generalization

7.5.1 Scale and Contrast Ratio Generalization. We train on problem scales N ∈[IN-DISTRIBUTION RANGE] and contrast ratios ρ ∈[IN-DISTRIBUTION RANGE], then test on held-out in-distribution (ID) and out-of-distribution (OOD) splits. OOD-scale tests go both smaller (N <[MIN TRAIN]) and larger (N >[MAX TRAIN]) than the training range; OOD-contrast tests use ρ outside the training distribution. We report the point at which performance degrades to match the next-best baseline as the "generalization boundary."

7.5.2 Problem Type Generalization. We train on [CLEAREST PROBLEM TYPE] and test on additional problem types: [LIST TYPES, e.g. heat equation, wave equation, elasticity, pressure Poisson on irregular domains]. For each, we report whether the preconditioner transfers without retraining, with fine-tuning, or requires retraining from scratch.

7.5.3 Precision and Breakdown. We vary PCG element precision (float32 / mixed float32+float64 accumulators / float64) and report solve time and achieved residual. We also identify the class of problems where our preconditioner fails to deliver iteration reduction over Jacobi and characterize the failure mode (high contrast ratio, pathological sparsity pattern, problem type outside training distribution).

7.6 Hardware Efficiency

We profile the best configuration at N =[MEDIUM SCALE] using `nsight-compute` (or `torch.profiler`). We report Tensor Core utilization, DRAM bandwidth, SM occupancy, and achieved FLOP/s as

Table 4. PCG solve performance across problem scales, [PROBLEM TYPE]. "Setup" = preconditioner construction time (ms); "Solve" = PCG solve time (ms) with CUDAGraph replay where supported; "Total" = Setup + Solve; "Iters" = PCG iterations; `rtol` = 10^{-6}. "—" = method failed to converge within [MAX ITERS] iterations. "*" = CUDAGraph not supported; times include Python dispatch overhead. Our inference time is the same as "Setup"; solve uses CUDAGraph. Best GPU total time per scale in **bold**.

	N =[SMALL]				N =[MEDIUM]				N =[LARGE]			
Method	Setup	Solve	Total	Iters	Setup	Solve	Total	Iters	Setup	Solve	Total	Iters
Unpreconditioned CG (GPU)	0	TODO	TODO	TODO	0	TODO	TODO	TODO	0	TODO	TODO	TODO
Jacobi (GPU)	<1	TODO	TODO	TODO	<1	TODO	TODO	TODO	<1	TODO	TODO	TODO
IC / MULTICOLOR_DILU (GPU)*	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO
AMG / AMGX (GPU)*	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO
Neural SPAI [Yang et al. 2025] (GPU)	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO
Ours (GPU)	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO
IC (CPU)	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO
AMG / PyAMG (CPU)	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO

Table 5. Generalization to out-of-distribution scales and contrast ratios. "ID" = in-distribution test; "OOD" = out-of-distribution. PCG iterations to `rtol` = 10^{-6}; "†" = failed to converge. Best per row in **bold**.

Split	N / ρ	Ours (iters)	Neural SPAI	IC	AMG
ID scale, ID ρ	[VAL] / [VAL]	TODO	TODO	TODO	TODO
OOD scale (smaller)	[VAL] / [ID ρ]	TODO	TODO	TODO	TODO
OOD scale (larger)	[VAL] / [ID ρ]	TODO	TODO	TODO	TODO
ID scale, OOD ρ (high)	[ID] / [HIGH]	TODO	TODO	TODO	TODO
ID scale, OOD ρ (extreme)	[ID] / [EXTREME]	TODO†	TODO†	TODO	TODO

Table 6. Cross-problem-type generalization. Model trained on [TRAINING TYPE] only, tested zero-shot on other types. Reported metric: PCG iterations to `rtol` = 10^{-6}. "FT" = result after [NUM STEPS] fine-tuning on 10% of target-domain training frames.

Test problem type	Ours (zero-shot)	Ours (FT)	Neural SPAI	IC
[TRAINING TYPE] (in-dist.)	TODO	—	TODO	TODO
[TYPE 2]	TODO	TODO	TODO	TODO
[TYPE 3]	TODO	TODO	TODO	TODO
[TYPE 4 / extreme / OOD]	TODO†	TODO	TODO†	TODO

Table 7. Effect of PCG floating-point precision at N =[MEDIUM SCALE], [PROBLEM TYPE]. "Elem." = element dtype for A, $\mathbf{b}$, iterates; "Acc." = scalar accumulator dtype. "Achieved `rtol`" = actual relative residual at convergence or [MAX ITERS] if not converged. Times in ms (GPU, CUDAGraph).

Elem.	Acc.	Solve (ms)	Iters	Achieved `rtol`
float32	float32	TODO	TODO	TODO
float32	float64 (mixed)	TODO	TODO	TODO
float64	float64	TODO	TODO	TODO

fractions of theoretical peak, for (a) the inference (forward) pass and (b) one PCG iteration (apply_block_diagonal_m_into + CSR SpMV).

7.7 Training Dynamics and Convergence

8 Limitations and Future Work

[To be completed.]

Table 8. GPU hardware utilization at N =[MEDIUM SCALE] on [GPU MODEL]. "TC util." = Tensor Core utilization (% of peak); "BW util." = DRAM bandwidth utilization; "Occ." = SM warp occupancy; "GFLOP/s" = achieved vs. theoretical peak. Profiled with `nsight-compute` / `torch.profiler`.

Operation	TC util. (%)	BW util. (%)	Occ. (%)
Inference (full forward)	TODO	TODO	TODO
PCG iter: CSR SpMV (A)	TODO	TODO	TODO
PCG iter: diag GEMM	TODO	TODO	TODO
PCG iter: off-diag BMM† (coarse)	TODO	TODO	TODO
PCG iter: projection/prolongation	TODO	TODO	TODO
Neural SPAI [Yang et al. 2025]: SpMV	TODO	TODO	TODO
IC / MULTICOLOR_DILU: trisol.	TODO	TODO	TODO

Fig. 7. Roofline analysis on [GPU MODEL] for per-PCG-iteration operations. Dashed lines show hardware-peak GFLOP/s (horizontal) and bandwidth-bound slope. Our batched GEMM operations (squares) are compute-bound; SpMV-based methods (circles) and IC triangular solves (triangles) are bandwidth-bound.

Fig. 8. Training dynamics of the best configuration on [PROBLEM TYPE], N =[SCALE]. *Left*: $\mathcal{L}_{\cos}$ (solid) and equivalent SAI loss value (dashed, secondary axis) vs. training step. Shaded region: ±1 std. across [NUM] seeds. *Right*: PCG iteration count (median ± IQR over validation frames) vs. training step. Horizontal lines show classical baselines.

9 Conclusion

We have presented a hierarchical transformer-based neural preconditioner anchored by an analytically constructed, weak-admissibility $\mathcal{H}$-matrix partition. By treating diagonal blocks with full-rank symmetric factors and off-diagonal blocks with learned low-rank factorizations at a fixed coarse resolution, we obtain a preconditioner that addresses both high-frequency local errors and low-frequency long-range errors in a unified $O(N)$ forward pass. The architecture processes all blocks in two compact transformer stacks over contiguous memory, achieving high GPU utilization and alignment

with modern hardware trends. We believe this represents a qualitatively new direction for neural preconditioning: moving away from sparsity-pattern constraints and toward full-graph representations whose structural bias is derived directly from the spectral properties of the underlying physics.

Acknowledgments

Omitted for anonymous review.

References

Michele Benzi, Carl D. Meyer, and Miroslav Tůma. 1996. A Sparse Approximate Inverse Preconditioner for the Conjugate Gradient Method. *SIAM Journal on Scientific Computing* 17, 5 (1996), 1135–1149.

Jie Chen. 2024. Graph Neural Preconditioners for Iterative Solutions of Sparse Linear Systems. *arXiv preprint arXiv:2406.00809* (2024). https://arxiv.org/abs/2406.00809 Published at ICLR 2025; see also Chen2025graph entry in this file..

Leslie Greengard and Vladimir Rokhlin. 1987. A Fast Algorithm for Particle Simulations. *J. Comput. Phys.* 73, 2 (1987), 325–348. doi:10.1016/0021-9991(87)90140-9

Wolfgang Hackbusch. 1999. *A Sparse Matrix Arithmetic Based on $\mathcal{H}$-Matrices. Part I: Introduction to $\mathcal{H}$-Matrices.* Springer, Berlin. Computing 62(2):89–108. Often cited as hierarchical matrix foundations..

Wolfgang Hackbusch and Boris N. Khoromskij. 2002. *Adaptive $\mathcal{H}$-Matrix Approximation on General Domains.* Springer. Survey of adaptive hierarchical matrix techniques..

Paul Häusner, Ozan Öktem, and Jens Sjölund. 2023. Neural Incomplete Factorization: Learning Preconditioners for the Conjugate Gradient Method. *arXiv preprint arXiv:2305.16368* (2023). https://arxiv.org/abs/2305.16368

Michael F. Hutchinson. 1989. A Stochastic Estimator of the Trace of the Influence Matrix for Laplacian Smoothing Splines. *Communications in Statistics—Simulation and Computation* 18, 3 (1989), 1059–1076.

L. Yu. Kolotilina and A. Yu. Yeremin. 1993. Factorized Sparse Approximate Inverse Preconditionings I: Theory. *SIAM J. Matrix Anal. Appl.* 14, 1 (1993), 45–58.

Yichen Li, Peter Yichen Chen, Tao Du, and Wojciech Matusik. 2023. Learning Preconditioners for Conjugate Gradient PDE Solvers. *arXiv preprint arXiv:2305.16432* (2023). https://arxiv.org/abs/2305.16432 Public drafts also used the name Neural PCG (ICLR 2023)..

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. 2020a. Multipole Graph Neural Operator for Parametric Partial Differential Equations. *Advances in Neural Information Processing Systems* 33 (2020).

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. 2020b. Neural Operator: Graph Kernel Network for Partial Differential Equations. *arXiv preprint arXiv:2003.03485* (2020). https://arxiv.org/abs/2003.03485

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. 2021. Fourier Neural Operator for Parametric Partial Differential Equations. In *International Conference on Learning Representations*. https://openreview.net/forum?id=c8P9NQVtmnO

Chih-Jen Lin and Jorge J. Moré. 1999. Incomplete Cholesky Factorizations with Limited Memory. *SIAM Journal on Scientific Computing* 21, 1 (1999), 24–45.

Lijun Liu, Shengguo Li, Xiangke Hu, Yutong Wang, Xuejun Liu, and Jingling Xue. 2016. Exploring Data Level Parallelism in Incomplete LU Factorization on GPUs. *IEEE Transactions on Parallel and Distributed Systems* 27, 12 (2016), 3397–3410.

Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. 2021. Learning Nonlinear Operators via DeepONet Based on the Universal Approximation Theorem of Operators. *Nature Machine Intelligence* 3, 3 (2021), 218–229.

Maxim Naumov, Michael Chien, Paul Vandermersch, Ujval Kapasi, Boris Catanzaro, and Michael Garland. 2015. cuSPARSE Library. In *GPU Technology Conference (GTC)*. NVIDIA AMGX / sparse linear algebra stack; widely referenced for GPU AMG and sparse solvers..

Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. 2019. Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations. *J. Comput. Phys.* 378 (2019), 686–707.

Kirill Rudikov, Anastasia Markeeva, Vasily Bulatov, and Dmitry Vetrov. 2024. Neural Functional Operator for Parametric PDEs. *arXiv preprint arXiv:2402.01030* (2024). https://arxiv.org/abs/2402.01030

John W. Ruge and Klaus Stüben. 1987. Algebraic Multigrid (AMG). *Multigrid Methods* 3 (1987), 73–130.

Yousef Saad. 1993. A Flexible Inner-Outer Preconditioned GMRES Algorithm. *SIAM Journal on Scientific Computing* 14, 2 (1993), 461–469.

Yousef Saad. 2003. *Iterative Methods for Sparse Linear Systems* (2 ed.). SIAM, Philadelphia, PA.

Konstantin Trifonov et al. 2025. Graph Neural Networks for Preconditioning Linear Systems. *arXiv preprint* (2025). Placeholder entry; replace with definitive citation when available..

Konstantin Trifonov and Stefan Güttel. 2024. Linear Algebraic Preconditioning using Machine Learning. *arXiv preprint arXiv:2403.11463* (2024). https://arxiv.org/abs/2403.11463 Verify final venue if published..

Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying Graph Convolutional Networks. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*. PMLR, 6861–6871. http://proceedings.mlr.press/v97/wu19e.html

Ichitaro Yamazaki, Stanimire Tomov, and Jack Dongarra. 2020. Mixed-Precision Cholesky QR Factorization and Its Parallelization on GPUs. *Parallel Comput.* 99 (2020), 102693.

Zherui Yang, Zhehao Li, Kangbo Lyu, Yixuan Li, Tao Du, and Ligang Liu. 2025. Learning Sparse Approximate Inverse Preconditioners for Conjugate Gradient Solvers on GPUs. *arXiv preprint arXiv:2510.27517* (2025). https://arxiv.org/abs/2510.27517

Received 30 April 2026; revised ; accepted